

神经网络优化器

发展历史 · 核心公式 · 深度对比 · Transformer / LLM 视角

从 SGD → Momentum → AdaGrad → RMSProp → Adam → AdamW → Shampoo / Sophia

主线：方向去噪 → 参数尺度自适应 → 预条件 / 曲率建模

技术学习笔记 · 适用于深度学习 / NLP / Transformer / LLM

目录 / 阅读路径

本笔记保留完整的优化器演化链，并按“基础 → Adam 家族 → 大模型优化 → LLM 实践 → 总结复习”组织。

部分	章节范围	核心主题
Part A	0-9	统一框架、时间线、SGD、Momentum、Nesterov、AdaGrad、RMSProp、AdaDelta
Part B	10-19	Adam、Bias Correction、Diagonal Preconditioning、AdamW、AMSGrad、RAdam、Nadam
Part C	20-29	Adafactor、LARS、LAMB、Shampoo、Lion、Sophia、Preconditioning、Newton
Part D	30-40	SGD vs Adam、Optimizer State、LLM AdamW、 β 、Batch Size、 ϵ 、Weight Decay、Clipping、Scheduler、Warmup
Part E	41-46	问题→改进总表、几何信息量、三大家族、横向比较、选择建议、核心直觉
Part F	47-54	速记、大模型系统视角、核心结论、公式索引、面试题、最终心智模型、一句话总结

推荐阅读路线：

首次系统学习：0 → 3 → 4 → 6 → 8 → 10 → 15 → 28 → 32 → 40 → 46

LLM 工程复习：10 → 15 → 20 → 31 → 32 → 33 → 35 → 37 → 38 → 39 → 40

面试速记：41 → 44 → 46 → 47 → 51 → 52 → 53

0. 总览：优化器到底在优化什么？

设神经网络参数为 θ_t ，损失函数为 $L(\theta)$ ，第 t 步梯度为 $g_t = \nabla_{\theta} L(\theta_t)$ 。所有一阶优化器都可以看成对“原始梯度”进行不同形式的变换，再决定参数更新。

$$g_t = \nabla_{\theta} L(\theta_t)$$
$$\theta_{t+1} = \theta_t - \Delta \theta_t$$

统一视角： $\theta_{t+1} = \theta_t - \eta_t P_t g_t$

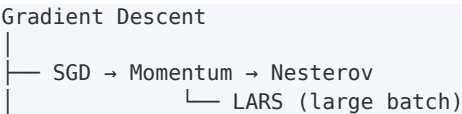
这里 η_t 是全局学习率， P_t 是预条件器（preconditioner）。SGD 的 $P_t = I$ ；AdaGrad / Adam 近似使用对角预条件；Shampoo 则进一步使用矩阵预条件。

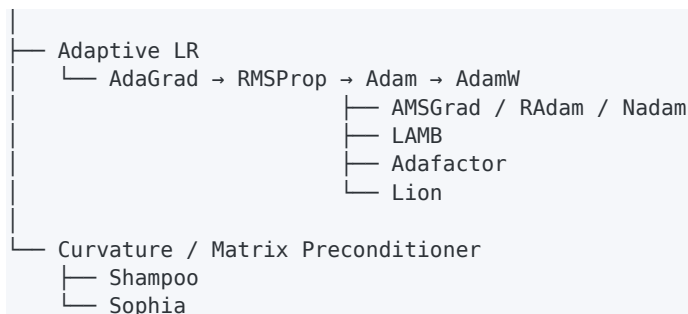
- 问题 1：梯度方向太 noisy，如何让方向更稳定？ → Momentum / Nesterov。
- 问题 2：不同参数的梯度尺度差异很大，如何给每个参数不同有效步长？ → AdaGrad / RMSProp / Adam。
- 问题 3：能否进一步利用参数维度之间的相关性或曲率？ → Shampoo / Sophia / 二阶方法。

优化器的发展，本质上是在越来越充分地利用 **loss landscape** 的几何信息。

1. 发展时间线

时间	优化器	核心思想	主要解决的问题
1950s+	Gradient Descent / SGD	沿负梯度方向更新	建立基本随机优化框架
1960s	Momentum	梯度加入惯性	SGD 振荡、噪声
1980s	Nesterov	在“预估位置”看梯度	Momentum 过冲
2011	AdaGrad	累计平方梯度	每个参数不同学习率
2012	RMSProp	平方梯度使用 EMA	AdaGrad 学习率衰减到接近 0
2012	AdaDelta	同时归一化梯度与更新尺度	减弱对手工 LR 的依赖
2014	Adam	Momentum + RMSProp	同时处理方向与尺度
2016	Nadam	Adam + Nesterov	更前瞻的动量
2017	LARS	Layer-wise scaling	超大 batch SGD
2018	AMSGrad	二阶矩上界单调化	修复 Adam 特定收敛问题
2018	Adafactor	分解二阶矩状态	降低 Adam 显存
2018	Shampoo	矩阵预条件	对角近似太粗
2019	AdamW	解耦 Weight Decay	Adam 中 L2 ≠ 真正 WD
2019	LAMB	Adam + layer trust ratio	超大 batch Transformer
2019	RAdam	Rectified variance	Adam 训练早期二阶统计不稳
2023	Lion	sign + momentum	减少 optimizer state
2023	Sophia	Hessian diagonal + clipping	更直接利用曲率信息





2. Gradient Descent: 所有优化器的起点

$$\theta_{t+1} = \theta_t - \eta g_t$$

梯度下降只做一件事：在当前参数位置计算损失函数的局部斜率，然后沿负梯度方向移动。问题在于神经网络损失面通常具有强烈的各向异性（anisotropy）：不同方向曲率可能相差几个数量级。

$$\lambda_1 \gg \lambda_2$$

- 学习率过大：高曲率方向可能振荡甚至发散。
- 学习率过小：低曲率方向推进非常慢。

因此，后续优化器的核心目标就是：在保持计算成本可接受的前提下，对方向、尺度和曲率进行更合理的校正。

3. SGD: 随机梯度下降

$$g_t = (1/B) \sum_{i \in B_t} \nabla_{\theta} L_i(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta g_t$$

Mini-batch SGD 用一个批次的梯度估计总体梯度，因此可以写成 $g_t = \nabla L(\theta_t) + \epsilon_t$ ，其中 ϵ_t 是 batch sampling noise。

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t) - \eta \epsilon_t$$

- 优点：实现简单、optimizer state 少、显存低；在很多视觉任务中最终泛化表现很强。
- 缺点：所有参数共享同一个基础学习率，无法自动处理不同坐标的梯度尺度。

示例： $g = [100, 0.001]^T$ ；同一个 η 很难同时适配两个方向。

4. Momentum: 给梯度加入“惯性”

$$v_t = \beta v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

另一种等价尺度写法是 $v_t = \beta v_{t-1} + (1-\beta)g_t$ 。常见 $\beta \approx 0.9$ 。展开后可看到过去梯度以指数衰减方式进入当前更新：

$$v_t = g_t + \beta g_{t-1} + \beta^2 g_{t-2} + \dots$$

在狭长谷底中，横向梯度如果长期同号会被累积；垂直方向若频繁正负交替则会被抵消。Momentum 因此可以视为对梯度序列做低通滤波（gradient low-pass filter）。

Momentum 主要解决“方向噪声”，而不是“每个参数学习率不同”。

5. Nesterov Momentum: 带预判的惯性

$$\begin{aligned}\theta_{\sim t} &= \theta_t - \eta \beta v_{t-1} \\ g_t &= \nabla L(\theta_{\sim t}) \\ v_t &= \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t\end{aligned}$$

普通 Momentum 在当前位置看梯度；Nesterov 先根据惯性“预测”将要到达的位置，再在那里计算梯度。这相当于让优化器提前感知可能的过冲。

6. AdaGrad: 真正的 per-parameter learning rate

$$\begin{aligned}s_t &= s_{t-1} + g_t^2 \\ \theta_{t+1} &= \theta_t - \eta \cdot g_t / (\sqrt{s_t} + \epsilon)\end{aligned}$$

所有平方、开方和除法都是逐元素（element-wise）的。累计平方梯度越大的参数，其有效学习率越小；很少被更新的参数则保留更大的有效步长。

$$\eta_i^{\text{eff}} = \eta / (\sqrt{s_{t,i}} + \epsilon)$$

这在稀疏特征和早期 NLP 中尤其有意义：高频词 embedding 被频繁更新，其步长逐渐减小；低频词仍可以获得较大的更新。

7. AdaGrad 的核心问题：学习率只会越来越小

$$s_t = \sum_{\tau=1}^t g_{\tau}^2$$

因为 s_t 单调累加，所以有效学习率 $\eta / \sqrt{s_t}$ 只会下降。在长周期非凸优化中，这会导致后期几乎停止学习。RMSProp 的核心改动就是“不要永久记住所有历史平方梯度”。

8. RMSProp: 用 EMA 取代永久累积

$$\begin{aligned}v_t &= \rho v_{t-1} + (1-\rho)g_t^2 \\ \theta_{t+1} &= \theta_t - \eta \cdot g_t / (\sqrt{v_t} + \epsilon)\end{aligned}$$

RMSProp 使用指数移动平均，只保留最近一段时间的梯度尺度信息。典型 $\rho \approx 0.9$ 。于是过去很久的大梯度不会永久压低当前学习率。

$$v_t = (1-\rho)(g_t^2 + \rho g_{t-1}^2 + \rho^2 g_{t-2}^2 + \dots)$$

RMSProp $\approx$ AdaGrad + Forget old gradients

9. AdaDelta: 进一步减弱对全局学习率的依赖

AdaDelta 在 RMSProp 的基础上，不仅对梯度平方做 EMA，也对历史参数更新量平方做 EMA，用“更新的 RMS / 梯度的 RMS”形成无量纲尺度。

$$\begin{aligned}E[g^2]_t &= \rho E[g^2]_{t-1} + (1-\rho)g_t^2 \\ \Delta\theta_t &= - \text{RMS}[\Delta\theta]_{t-1} / \text{RMS}[g]_t \cdot g_t \\ E[\Delta\theta^2]_t &= \rho E[\Delta\theta^2]_{t-1} + (1-\rho)\Delta\theta_t^2\end{aligned}$$

它在历史上重要，但在现代 Transformer 中已远不如 AdamW 常见。

10. Adam: Momentum + RMSProp

Adam 的直觉非常简洁：一阶矩负责“方向”，二阶矩负责“尺度”。

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$$

m_t 近似梯度的一阶矩 $E[g]$ ； v_t 近似未中心化二阶矩 $E[g^2]$ 。注意 v_t 并不是严格统计学方差 $\text{Var}(g)=E[g^2]-E[g]^2$ 。

11. Adam 的 Bias Correction

Adam 通常以 $m_0=v_0=0$ 初始化，因此训练早期两个 EMA 会系统性偏向 0。例如 $\beta_1=0.9$ 时，第一步 $m_1=0.1g_1$ 。

$$\hat{m}_t = m_t / (1-\beta_1^t)$$

$$\hat{v}_t = v_t / (1-\beta_2^t)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

12. Adam 为什么强：坐标重缩放 (Diagonal Preconditioning)

假设一个坐标长期梯度约为 100，而另一个约为 0.001。Adam 的二阶矩会分别把它们除以接近自身 RMS 的尺度，因此大幅压缩坐标间的梯度幅度差异。

若 $g \approx 100$ ，则 $m \approx 100$ ， $\sqrt{v} \approx 100$ ，因此 $m/\sqrt{v} \approx 1$

若 $g \approx 0.001$ ，则 $m \approx 0.001$ ， $\sqrt{v} \approx 0.001$ ，因此 $m/\sqrt{v} \approx 1$

这并不意味着每个坐标更新完全相同，因为方向变化、EMA、 ϵ 和 bias correction 都会影响结果；但它解释了 Adam 为什么对强烈 anisotropic 的参数空间特别友好。

13. 为什么 Adam 特别适合 Transformer / LLM?

Transformer 同时包含 Embedding、Q/K/V 投影、Attention output、MLP Up/Down Projection、Norm 参数以及输出头，它们的 fan-in、激活尺度和梯度统计差异明显。

$$\eta_i^{\text{eff}} \approx \eta / (\sqrt{v_i} + \epsilon)$$

- SGD：只有一个全局基础尺度，很依赖精细调参。
- Adam：按参数坐标动态调节有效步长，同时使用动量平滑方向。
- 因此 Transformer / LLM 预训练长期以 Adam family，尤其 AdamW，作为强默认基线。

14. Adam 的关键问题：L2 Regularization $\neq$ Weight Decay

在 SGD 中，对损失加入 L2 项：

$$L' = L + (\lambda/2)\|\theta\|^2$$

$$g' = g + \lambda\theta$$

$$\text{SGD: } \theta \leftarrow \theta - \eta(g + \lambda\theta) = (1 - \eta\lambda)\theta - \eta g$$

因此在普通 SGD 中，L2 regularization 与 multiplicative weight decay 可以等价。但 Adam 会对 $g + \lambda\theta$ 一起进行自适应归一化，于是正则项也被 $1/\sqrt{v}$ 缩放，不再等于统一的参数衰减。

15. AdamW: 把 Weight Decay 从自适应梯度中解耦

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)g_t \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2)g_t^2 \\ \theta_{t+1} &= (1-\eta\lambda)\theta_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) \end{aligned}$$

AdamW 保留 Adam 的梯度更新，但把 weight decay 直接施加在参数上，不让他经过 adaptive normalization。

AdamW = Adam + Decoupled Weight Decay

16. Adam vs AdamW: 数据流对比

Adam + L2:
gradient $\rightarrow g + \lambda\theta \rightarrow$ momentum / second moment $\rightarrow$ adaptive normalization $\rightarrow$ update

AdamW:
gradient $\rightarrow$ Adam adaptive update
parameter $\rightarrow$ independent multiplicative decay

$$\text{AdamW: } \theta \leftarrow \theta - \eta u - \eta \lambda \theta, \quad u = \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

17. AMSGrad: 约束二阶矩不要回落

Adam 的 EMA 二阶矩 v_t 可能下降，从而导致某些坐标的有效学习率突然增大。AMSGrad 维护历史最大二阶矩：

$$\begin{aligned} \tilde{v}_t &= \max(\tilde{v}_{t-1}, v_t) \\ \theta_{t+1} &= \theta_t - \eta \cdot \hat{m}_t / (\sqrt{\tilde{v}_t} + \epsilon) \end{aligned}$$

它的重要性更多来自理论收敛修正；在实际 Transformer 训练里，AdamW 的普及程度明显更高。

18. RAdam: 修正 Adam 训练早期的二阶统计不稳定

$$\begin{aligned} \rho_\infty &= 2/(1-\beta_2) - 1 \\ \rho_t &= \rho_\infty - 2t\beta_2^t/(1-\beta_2^t) \end{aligned}$$

训练早期 v_t 的统计样本不足，自适应缩放可能方差很大。RAdam 根据二阶矩估计的“有效自由度”决定是否应用完整 adaptive scaling，并加入 rectification factor。可以把它理解为一种对 Adam 早期阶段的自动稳定机制。

19. Nadam: Nesterov + Adam

Nadam 将 Nesterov lookahead 思路用于 Adam 的一阶动量，使一阶方向更“前瞻”。概念上：Adam = Momentum + RMSProp；Nadam = Nesterov Momentum + RMSProp。实践中仍未取代 AdamW。

20. Adafactor: 解决 Adam optimizer state 太大

Adam 对每个参数维护 m 和 v 两个与参数同形状的状态。对超大模型来说，optimizer state 本身就可能成为主要显存/内存成本。Adafactor 的核心是对矩阵参数的二阶矩进行低秩式因子分解。

$$\begin{aligned} W &\in \mathbb{R}^{n \times m}; \text{ Adam 保存 } V \in \mathbb{R}^{n \times m} \\ \text{Adafactor 保存 } R &\in \mathbb{R}^n, C \in \mathbb{R}^m: nm \rightarrow n+m \end{aligned}$$

例如 4096×4096 矩阵完整二阶矩有 16,777,216 个元素，而行列统计只需要 8192 个元素。

21. Adafactor 的近似公式

$$\begin{aligned} S_t &= G_t \odot G_t \\ R_t &= \beta_2 R_{t-1} + (1-\beta_2)(1/m) S_t 1_m \\ C_t &= \beta_2 C_{t-1} + (1-\beta_2)(1/n) 1_n^T S_t \\ \hat{V}_t &\approx (R_t C_t) / \text{mean}(R_t) \\ U_t &= G_t / (\sqrt{\hat{V}_t} + \epsilon) \end{aligned}$$

它牺牲了一部分二阶统计精度，换取显著降低的 optimizer state，因而特别适合大矩阵占主导的 Transformer。

22. LARS: 超大 Batch SGD 的 Layer-wise Scaling

$$\begin{aligned} r_l &= \|w_l\| / (\|g_l\| + \lambda \|w_l\| + \epsilon) \\ \eta_l &= \eta r_l, \quad w_l \leftarrow w_l - \eta_l g_l \end{aligned}$$

大 batch 训练中，不同层的参数范数与梯度范数差异可能非常大。LARS 用 trust ratio 控制每层相对更新尺度，使 $\|\Delta w_l\| / \|w_l\|$ 处在合理范围。

23. LAMB: 把 LARS 的 trust ratio 搬到 Adam

$$\begin{aligned} u_t &= \hat{m}_t / (\sqrt{\hat{V}_t} + \epsilon) + \lambda \theta_t \\ r_t &= \|\theta_t\| / \|u_t\| \\ \theta_{t+1} &= \theta_t - \eta r_t u_t \end{aligned}$$

LAMB = Adam-style adaptive update + Layer-wise Trust Ratio

其历史价值主要体现在超大 batch 的 BERT / Transformer 训练。

24. Shampoo: 从对角预条件走向矩阵预条件

Adam 只保存每个坐标自己的二阶尺度，不刻画参数维度之间的相关性。Shampoo 针对矩阵参数维护左右两个二阶统计矩阵：

$$\begin{aligned} L_t &= L_{t-1} + G_t G_t^T \\ R_t &= R_{t-1} + G_t^T G_t \\ \tilde{G}_t &= L_t^{-1/4} G_t R_t^{-1/4} \\ W_{t+1} &= W_t - \eta \tilde{G}_t \end{aligned}$$

相比 Adam 的 diagonal geometry，Shampoo 能捕获维度间相关结构，但代价是矩阵统计、逆矩阵根、通信和内存明显增加。

25. Lion: 只保留动量，不保存 second moment

$$\begin{aligned} c_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\ \theta_t &\leftarrow \theta_t - \eta \text{sign}(c_t) \end{aligned}$$

$$m_t = \beta_2 m_{t-1} + (1-\beta_2)g_t$$

Lion 的更新幅度主要由全局学习率决定，各坐标主要通过 sign 决定方向；相比 Adam 少保存一个 v 状态，因此 optimizer state 更低。若采用解耦 weight decay，可再加入 $(1-\eta\lambda)\theta$ 。

26. Sophia：开始明确利用 Hessian 对角信息

Newton 方法理想更新为 $\Delta\theta = -H^{-1}g$ ，其中 $H = \nabla^2 L(\theta)$ 。它能直接校正曲率，但对大模型构造完整 Hessian 完全不可行。Sophia 采用近似对角 Hessian 信息，并对更新做 clipping。

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$$

$$h_t = \beta_2 h_{t-1} + (1-\beta_2)\hat{h}_t$$

$$\theta \leftarrow \theta - \eta \cdot \text{clip}(m_t / (\gamma h_t + \epsilon), -1, 1)$$

当某些方向曲率估计很小， g/h 可能非常大，因此 clipping 起到 trust-region-like 的保护作用。

27. 从 SGD 到 Sophia：优化器“知道”的信息越来越多

层级	代表方法	利用的信息	直觉
Level 0	SGD	当前梯度 g_t	只知道局部下降方向
Level 1	Momentum	$\text{EMA}(g)$	知道近期稳定方向
Level 2	RMSProp	$\text{EMA}(g^2)$	知道每个坐标的尺度
Level 3	Adam / AdamW	$\text{EMA}(g) + \text{EMA}(g^2)$	方向 + 坐标尺度
Level 4	Shampoo	$GG^T / G^T G$	利用矩阵维度相关性
Level 5	Sophia / Newton-like	Hessian / Hessian diagonal	更直接利用曲率

28. 统一视角：Preconditioning

$$\theta_{t+1} = \theta_t - \eta P_t g_t$$

方法	P_t / 等价作用
SGD	I
AdaGrad	$\text{diag}(1/(\sqrt{\sum g^2} + \epsilon))$
RMSProp	$\text{diag}(1/(\sqrt{\text{EMA}(g^2)} + \epsilon))$
Adam	对 momentum m_t 使用 diagonal preconditioner
Shampoo	结构化矩阵 preconditioner
Newton	H^{-1}

从这个角度看，“换优化器”并不是换一种神秘更新公式，而是在选择不同复杂度的局部坐标系。

29. 为什么不直接一直用 Newton?

$$\theta_{t+1} = \theta_t - H^{-1}g$$

- 完整 Hessian 对 N 参数需要 $O(N^2)$ 存储，百亿参数量级完全不可行。

- 显式求逆或精确矩阵分解成本更高。
- 神经网络是非凸问题，Hessian 可能有负特征值，Newton direction 不一定是下降方向。

现代优化器的核心目标：尽量获得二阶 geometry 的收益，同时保持接近一阶优化的成本。

30. SGD + Momentum vs Adam / AdamW

维度	SGD + Momentum	Adam / AdamW
一阶动量	✓	✓
二阶梯度统计	✗	✓
Per-parameter LR	✗	✓
Optimizer state	低	高
超参数敏感度	通常更高	通常更容易起步
前期收敛速度	较慢	通常更快
稀疏梯度	一般	较好
Transformer / LLM	较难调	主流
CV	长期强基线	同样常用
泛化	某些任务更好	依任务而定

需要区分 optimization performance 与 generalization performance：更快降低训练 loss，不等于最终测试泛化一定更好。

31. Optimizer State 的显存 / 内存成本

优化器	主要额外状态	相对特征
SGD	几乎无额外 optimizer state	最低
Momentum SGD	1× 参数大小的 momentum	低
Adam / AdamW	m + v, 约 2× 参数大小	高
Lion	主要 1× momentum	低于 Adam
Adafactor	矩阵二阶矩 factorized	大矩阵上显著降低
Shampoo	矩阵统计 / 预条件状态	可能很高

因此超大模型中，optimizer choice 不只是数学问题，也是内存、分布式通信与系统设计问题。

32. 为什么 LLM 训练通常默认 AdamW?

- Transformer 参数种类多、梯度尺度差异大。
- 训练 loss landscape 强烈 anisotropic。
- 训练初期对更新尺度非常敏感。
- AdamW 同时提供 momentum、per-parameter normalization、bias correction 与 decoupled weight decay。

- 它与 warmup、cosine / linear decay、gradient clipping 组合成熟。

典型 LLM 基线：AdamW + Warmup + LR Decay + Gradient Clipping

33. 一个典型的 LLM AdamW 配置

$\beta_1 \approx 0.9$

$\beta_2 \approx 0.95 \sim 0.999$

$\epsilon \approx 10^{-8}$ （具体取值依实现与数值精度）

weight decay $\lambda \approx 0.01 \sim 0.1$ （依模型与配方而变化）

LLM 训练中经常把 β_2 从经典 Adam 默认 0.999 调低到 0.95 左右，使 second moment 更快适应新的梯度统计。具体最优值仍需结合 batch size、训练 token 预算和学习率共同调节。

34. β_1 、 β_2 的“有效记忆长度”

EMA: $m_t = \beta m_{t-1} + (1-\beta)x_t$

粗略有效窗口: $T \approx 1/(1-\beta)$

β	粗略有效窗口 T	直觉
0.9	≈ 10 steps	短期方向
0.99	≈ 100 steps	较长统计
0.999	≈ 1000 steps	非常平滑的二阶尺度

因此 Adam 常见 $\beta_1=0.9$ 可以理解为“方向看短期”， β_2 较大则表示“尺度看更长历史”。

35. Batch Size $\times \beta$ ：一个经常被忽略的关系

$N_{\text{samples}} \approx B/(1-\beta)$

如果 batch size 为 B，而 EMA 有效窗口约 $1/(1-\beta)$ ，那么二阶统计实际覆盖的样本量大约与 $B/(1-\beta)$ 成正比。例如 $B=1024$ 、 $\beta_2=0.999$ 时，相当于整合约百万级样本范围的历史信号。

所以改变 batch size 后，即使 β 不变，optimizer dynamics 也已经改变。

36. ϵ 不只是“防止除零”

Adam update $\propto m / (\sqrt{v} + \epsilon)$

当 $\sqrt{v} \ll \epsilon$ 时，更新近似变成 m/ϵ ，此时 ϵ 主导有效尺度。因此 ϵ 既是数值稳定项，也会改变优化行为。在低精度训练、很小梯度、Normalization-heavy 网络中尤其值得关注。

37. Weight Decay 的参数分组 Gotcha

实际 Transformer 训练通常不会对所有参数统一做 weight decay。常见规则是：

参数类型	常见策略
Linear / Projection weight	Decay
Bias	No decay
LayerNorm / RMSNorm scale	No decay
Embedding / output weight	依具体配方决定

因此实现中经常构造 decay_params 与 no_decay_params 两个 parameter groups。

38. Gradient Clipping 与 Optimizer 解决的是不同问题

若 $\|g\|_2 > C$, 则 $g \leftarrow g \cdot C/\|g\|_2$

机制	主要解决的问题
Gradient clipping	单次异常梯度过大 / 梯度爆炸
Momentum	时间维度的方向噪声与振荡
Adam / RMSProp	长期不同参数坐标的尺度不匹配
Weight decay	参数幅度与正则化

Gradient clipping 不是 Adam 的替代品；Adam 也不是 gradient clipping 的替代品。

39. Adam 已经“自适应”，为什么还需要 LR Scheduler?

$\eta_{\{i,t\}}^{\text{eff}} = \eta_t / (\sqrt{\hat{v}_{\{i,t\}} + \epsilon})$

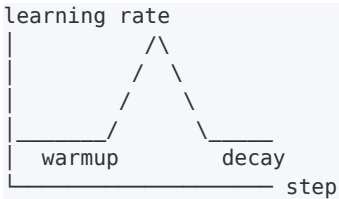
Adam 的 v_i 主要控制参数之间的相对尺度，而 η_t 仍控制整个训练过程随时间变化的全局步长。

- Adam / RMSProp: parameter-wise relative scale。
- LR scheduler: training-time global scale。

40. 为什么 Transformer / LLM 常用 Warmup?

$\eta_t = \eta_{\text{max}} \cdot t/T_{\text{warmup}}, \quad t \leq T_{\text{warmup}}$

训练最开始时 m_t 、 v_t 尚未稳定，模型激活和梯度统计也在快速变化。如果直接使用峰值学习率，可能造成过大的参数更新。Warmup 先逐步放大学习率，再进入稳定训练和 decay 阶段。



41. “前一代问题 → 下一代改进” 总表

优化器	前一代的主要问题	核心改进
SGD	基本可扩展优化	Mini-batch stochastic gradients
Momentum	SGD 抖动 / 方向噪声	历史梯度 EMA
Nesterov	Momentum 容易过冲	Lookahead gradient
AdaGrad	统一 LR 不适合所有参数	Per-parameter LR
RMSProp	AdaGrad LR 只降不升	Second moment EMA
AdaDelta	仍较依赖手工 LR	用历史更新尺度归一
Adam	RMSProp 缺少方向 momentum	First + second moment
AMSGrad	Adam 有效 LR 可能异常增加	max second moment
AdamW	L2 与 WD 被 adaptive scaling 混合	解耦 weight decay
RAdam	Adam early variance 不稳	Rectification
LARS	大 batch SGD 层间尺度不匹配	Layer trust ratio
LAMB	大 batch Adam 层间尺度不匹配	Adam + trust ratio
Adafactor	Adam state 太大	Factorized second moment
Lion	m+v 状态较重	只保留 momentum + sign
Shampoo	Adam 只看 diagonal	Matrix preconditioning
Sophia	Adam 不直接利用曲率	Approx. Hessian diagonal

42. 按“历史 / 几何信息量”重新分类

Level 0: g_t → SGD
 Level 1: $\text{EMA}(g)$ → Momentum
 Level 2: $\text{EMA}(g^2)$ → RMSProp
 Level 3: $\text{EMA}(g) + \text{EMA}(g^2)$ → Adam / AdamW
 Level 4: $GG^T, G^T G$ → Shampoo
 Level 5: $\nabla^2 L / \text{diag}(H)$ → Newton / Sophia-like

$g \rightarrow E[g] \rightarrow E[g^2] \rightarrow \text{matrix statistics} \rightarrow H$

43. 深度学习优化器的三大家族

43.1 Momentum family: 解决方向噪声

SGD → Momentum → Nesterov

43.2 Adaptive / Diagonal family: 解决坐标尺度

AdaGrad → RMSProp → Adam → AdamW
 ├─ AMSGrad / RAdam / Nadam
 ├─ Adafactor / LAMB
 └─ Lion

43.3 Curvature / Preconditioner family: 解决 loss geometry

Newton / natural-gradient ideas
 ↓ approximate structure
 Shampoo / Sophia / other second-order-like methods

44. 最终横向比较

Optimizer	Momentum	Adaptive Scale	Layer Scale	曲率 / 结构	State	Transformer
SGD	✗	✗	✗	✗	极低	较弱
Momentum	✓	✗	✗	✗	低	一般
Nesterov	✓	✗	✗	✗	低	一般
AdaGrad	✗	✓	✗	对角统计	中	一般
RMSProp	✗	✓	✗	对角统计	中	好
Adam	✓	✓	✗	对角统计	高	很好
AdamW	✓	✓	✗	对角统计	高	主流
AMSGrad	✓	✓	✗	对角统计	高	可用
Adafactor	可选	✓	✗	Factored	低	很好
LARS	✓	✗	✓	✗	低	特定场景
LAMB	✓	✓	✓	对角统计	高	大 Batch
Lion	✓	sign	✗	✗	中	很好
Shampoo	✓	✓	✓	矩阵	很高	研究 / 大规模
Sophia	✓	✓	✗	Hessian diagonal	高	Transformer 方向

45. 实际选择建议

场景	优先考虑
普通深度学习项目，缺少先验	AdamW 作为强默认基线
CNN / 追求最终泛化	SGD + Momentum 与 AdamW 都值得实验
超大模型、optimizer memory 是瓶颈	Adafactor / Lion / optimizer-state sharding
超大 batch	AdamW / LAMB, SGD 系可看 LARS
研究结构化预条件	Shampoo
研究显式曲率	Sophia-like / second-order methods
Transformer / LLM 标准 baseline	AdamW + scheduler + clipping

真正值得优先调节的通常不是“不断换优化器名字”，而是学习率、warmup、decay schedule、 β_1 、 β_2 、weight decay、batch size、gradient clipping 之间的联动。

46. 最值得形成的直觉

优化器	它真正问的问题
SGD	梯度告诉我往哪里走？
Momentum	最近一直往哪里走？
AdaGrad / RMSProp	每个方向过去的梯度尺度有多大？
Adam	长期方向是什么，同时每个方向应该缩放多少？

优化器	它真正问的问题
AdamW	优化与参数正则化能否分开？
Shampoo	不同参数方向之间是否相关？
Sophia / Newton	loss surface 在这里到底有多弯？

最终目标：寻找成本可接受的 $P_t \approx H_t^{-1}$

$$\Delta\theta_t = -\eta P_t g_t$$

不仅要知道“往哪里走”，还要知道“这个空间是什么形状”。

47. 一页速记：从 SGD 到 AdamW

```

SGD
 $\theta \leftarrow \theta - \eta g$ 
  ↓ 解决方向振荡
Momentum
 $v \leftarrow \beta v + g; \theta \leftarrow \theta - \eta v$ 
  ↓ 每个参数需要不同尺度
AdaGrad
 $s \leftarrow s + g^2; \theta \leftarrow \theta - \eta g / \sqrt{s}$ 
  ↓ 不要永久累积
RMSProp
 $v \leftarrow \rho v + (1-\rho)g^2; \theta \leftarrow \theta - \eta g / \sqrt{v}$ 
  ↓ 加入方向动量
Adam
 $m \leftarrow \text{EMA}(g); v \leftarrow \text{EMA}(g^2); \theta \leftarrow \theta - \eta \hat{m} / \sqrt{\hat{v}}$ 
  ↓ 正确处理权重衰减
AdamW
 $\theta \leftarrow (1-\eta\lambda)\theta - \eta \hat{m} / \sqrt{\hat{v}}$ 

```

48. 一页速记：从 AdamW 到大模型优化

```

AdamW
├─ optimizer state 太大 → Adafactor / Lion / ZeRO-style sharding
├─ 超大 batch 层间尺度差异 → LAMB
├─ diagonal preconditioner 太粗 → Shampoo
└─ 想显式利用曲率 → Sophia-like methods

```

49. 训练系统视角：优化器不是孤立模块

在真实 LLM 训练中，优化器与以下系统组件强耦合：

- 混合精度（BF16 / FP16 / FP8）与 loss scaling。
- Gradient accumulation 与 global batch size。
- 数据并行 / 张量并行 / Pipeline 并行。
- ZeRO / FSDP optimizer-state sharding。
- 梯度裁剪、权重衰减参数分组。
- Warmup、Cosine / Linear / WSD 等学习率调度。
- Checkpoint 中 optimizer state 的存储与恢复。

因此在大模型中，“optimizer 是否更好”必须同时考虑 convergence、tokens-to-quality、GPU memory、通信开销、checkpoint 体积与容错成本。

50. 核心结论

1. 优化器发展主线不是名称堆叠，而是从方向去噪、坐标尺度归一，逐步走向更丰富的几何 / 曲率建模。
2. Momentum 主要处理时间维度的梯度方向噪声；AdaGrad / RMSProp / Adam 主要处理参数坐标尺度。
3. Adam 的本质是 “first-moment direction + second-moment diagonal preconditioning”。
4. AdamW 的关键不是更复杂的 Adam，而是把 weight decay 从 adaptive gradient 中解耦。
5. Adafactor / Lion 反映了大模型时代的新目标：optimizer state 也是一等公民。
6. Shampoo / Sophia 代表继续向更强 preconditioning / curvature awareness 演进的方向。
7. 对 Transformer / LLM，AdamW 是极强默认 baseline；真正性能来自 optimizer、scheduler、batch、clipping、weight decay 和数值精度的联合设计。

51. 公式索引

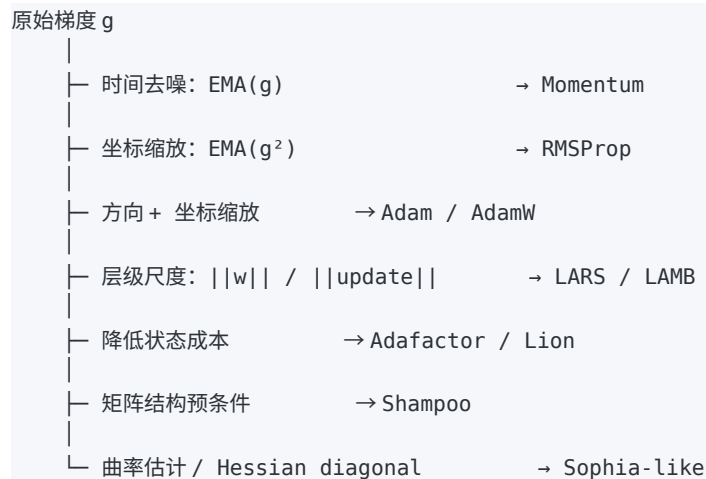
方法	核心更新公式
SGD	$\theta \leftarrow \theta - \eta g$
Momentum	$v \leftarrow \beta v + g; \theta \leftarrow \theta - \eta v$
Nesterov	在 lookahead $\theta_{\sim}$ 上计算梯度
AdaGrad	$s \leftarrow s + g^2; \theta \leftarrow \theta - \eta g / (\sqrt{s} + \epsilon)$
RMSProp	$v \leftarrow \rho v + (1 - \rho) g^2; \theta \leftarrow \theta - \eta g / (\sqrt{v} + \epsilon)$
Adam	$m \leftarrow \text{EMA}(g), v \leftarrow \text{EMA}(g^2), \theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$
AdamW	$\theta \leftarrow (1 - \eta \lambda) \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$
AMSGrad	$\tilde{v} \leftarrow \max(\tilde{v}, v)$
LARS	$r = \ w\ / (\ g\ + \lambda \ w\ + \epsilon)$
LAMB	$r = \ \theta\ / \ u\ , \theta \leftarrow \theta - \eta r u$
Shampoo	$\tilde{G} = L^{-1/4} G R^{-1/4}$
Sophia-like	$\theta \leftarrow \theta - \eta \text{clip}(m / (\gamma h + \epsilon), -1, 1)$

52. 面试 / 复习时最应该能回答的 10 个问题

1. 为什么 Momentum 能减少狭长谷底中的横向振荡？
2. AdaGrad 为什么特别适合稀疏特征，又为什么不适合很长训练周期？
3. RMSProp 相比 AdaGrad 改了哪个统计量？
4. Adam 的一阶矩、二阶矩分别是什么？为什么需要 bias correction？
5. 为什么说 Adam 是 diagonal preconditioner？
6. 为什么 Adam 中 L2 regularization 不等于 weight decay？AdamW 如何修复？
7. 为什么 Adam 已经自适应，LLM 仍然需要 warmup 和 LR scheduler？
8. β_2 从 0.999 改到 0.95 会对 second-moment 响应速度产生什么影响？
9. Adafactor、Lion 分别从哪里节省 optimizer state？

10. Shampoo / Sophia 相比 Adam 多利用了什么几何或曲率信息？

53. 最终心智模型



54. 一句话总结

SGD 只看坡度；Momentum 记住惯性；Adam 校正每个坐标的尺度；AdamW 再把正则化解耦；Shampoo / Sophia 继续尝试理解更真实的损失几何。

— End —